

Assistant Articles Multilingue

Documentation fichier par fichier — rôle, outils utilisés et fonctions clés

Dossier : 1-Agno

Ce document détaille, **fichier par fichier**, ce que fait chaque partie du code et quelles bibliothèques y sont utilisées — et pourquoi, précisément à cet endroit du code.

Vue d'ensemble des 3 fichiers

Fichier	Rôle en une phrase
rag_engine.py	Le moteur : extraction PDF, découpage du texte, recherche par similarité, agent IA.
app.py	L'interface : la page Streamlit que l'utilisateur voit et avec laquelle il interagit.
export_chat.py	L'export : transforme l'historique de conversation en fichier PDF ou Word téléchargeable.

Les trois fichiers se répartissent le travail dans cet ordre : **app.py** reçoit les actions de l'utilisateur et appelle **rag_engine.py** pour indexer les articles et répondre aux questions, puis appelle **export_chat.py** quand l'utilisateur veut télécharger la conversation.

rag_engine.py

Ce fichier ne contient **aucun élément visuel** : c'est la logique pure qui sait lire un PDF, découper un texte, retrouver les passages pertinents et interroger le modèle de langage.

Outils utilisés dans ce fichier

Import / bibliothèque	À quoi ça sert précisément dans ce fichier
os	Lit la variable d'environnement GROQ_API_KEY (la clé API), chargée depuis le fichier .env.
re (regex)	Dans chunk_text() : remplace tous les espaces/retours à la ligne successifs par un seul espace, pour aplatir le texte avant de le découper en blocs.
typing.Optional	Annotation de type pour indiquer que self.index peut valoir None tant qu'aucun article n'a été ajouté.
pymupdf (fitz)	Dans extract_text_from_pdf() : ouvre le PDF en mémoire et extrait le texte de chaque page. Choisi pour sa vitesse (~20x plus rapide que pypdf, utilisé dans la 1ère version).
sentence-transformers	Charge le modèle paraphrase-multilingual-MiniLM-L12-v2 qui transforme un texte (français, arabe ou anglais) en vecteur numérique — c'est le cœur de la recherche par sens plutôt que par mot-clé.
faiss	Dans la classe KnowledgeBase : stocke tous les vecteurs dans un IndexFlatL2 et retrouve, pour une question donnée, les vecteurs les plus proches en quelques millisecondes.
numpy	Convertit le vecteur de la question en tableau numpy, le format attendu par FAISS pour lancer une recherche.
python-dotenv	load_dotenv() charge le contenu du fichier .env au démarrage, pour récupérer la clé Groq sans l'écrire en dur dans le code.
agno (Agent)	Dans build_agent() : crée l'agent conversationnel et le relie à notre fonction de recherche maison via knowledge_retriever.
agno.models.groq (Groq)	Connecteur vers l'API Groq — précise quel modèle utiliser (llama-3.3-70b-versatile) pour générer les réponses.

Fonctions et classe principales

- load_embedder() — instancie le modèle d'embeddings multilingue.
- extract_text_from_pdf(file) — retourne le texte brut d'un PDF fourni (upload Streamlit ou fichier ouvert en local).
- chunk_text(text, source) — découpe un texte en blocs d'environ 900 caractères avec 150 caractères de chevauchement, alignés sur les mots (pas de coupure en plein milieu d'un mot).
- Classe KnowledgeBase — la mémoire de l'application :

- `add_document(text, source)` : découpe le texte, encode uniquement les *nouveaux* blocs et les ajoute à l'index FAISS existant.
- `remove_source(source) / clear()` : retirent un article (ou tout) et reconstruisent l'index.
- `search(query, num_documents)` : encode la question et retourne les blocs de texte les plus proches par le sens.
- `build_agent(knowledge_base)` — construit l'agent Agno relié à Groq, avec des instructions qui l'obligent à répondre dans la langue de la question et à rester fidèle aux articles fournis.

app.py

C'est le fichier que l'on lance (`streamlit run app.py`) : il construit toute la page web et appelle les fonctions de `rag_engine.py` et `export_chat.py` en réaction aux actions de l'utilisateur.

Outils utilisés dans ce fichier

Import / bibliothèque	À quoi ça sert précisément dans ce fichier
<code>streamlit</code>	Construit toute l'interface : <code>st.sidebar</code> (barre latérale pour ajouter des articles), <code>st.text_area/st.file_uploader</code> (saisie/import), <code>st.chat_message/st.chat_input</code> (fil de discussion), <code>st.download_button</code> (export), <code>st.session_state</code> (mémorise la base de connaissances et l'historique entre deux clics, puisque Streamlit ré-exécute tout le script à chaque interaction), et <code>st.cache_resource</code> (garde le modèle d'embeddings chargé une seule fois en mémoire pour toute la session).
<code>rag_engine</code> (module du projet)	Importe <code>KnowledgeBase</code> , <code>build_agent</code> , <code>extract_text_from_pdf</code> , <code>load_embedder</code> et <code>GROQ_API_KEY</code> — c'est tout le moteur RAG décrit page précédente, utilisé ici sans réimplémenter la logique.
<code>export_chat</code> (module du projet)	Importe <code>build_pdf</code> et <code>build_docx</code> , appelées quand l'utilisateur clique sur un bouton de téléchargement.

Organisation de la page

- **Barre latérale** — zone pour coller un article, importateur de fichiers PDF/TXT, liste des articles indexés avec suppression individuelle, boutons « Tout effacer » et boutons d'export.
- **Zone principale** — historique de conversation façon messagerie (`st.chat_message`) et champ de saisie de question (`st.chat_input`).
- Un `st.spinner(...)` s'affiche pendant l'indexation d'un article et pendant la recherche d'une réponse, pour que l'attente ne paraisse pas figée.
- Après chaque réponse, `st.rerun()` force un nouvel affichage immédiat, pour que les boutons d'export apparaissent dès que le premier message existe (sans ce `rerun()`, ils n'apparaîtraient qu'au clic suivant).

export_chat.py

Ce fichier transforme la liste des messages de la conversation (stockée dans `st.session_state.messages`) en un vrai fichier PDF ou Word prêt à être téléchargé — avec une gestion particulière du texte arabe, que ces bibliothèques ne traitent pas correctement par défaut.

Outils utilisés dans ce fichier

Import / bibliothèque	À quoi ça sert précisément dans ce fichier
<code>io</code>	Garde le fichier généré en mémoire (<code>BytesIO</code>) au lieu de l'écrire sur le disque, pour le transmettre directement au bouton de téléchargement de Streamlit.
<code>datetime</code>	Ajoute la date et l'heure d'export en en-tête du document.
<code>python-docx</code> (<code>Document</code> , <code>WD_ALIGN_PARAGRAPH</code> , <code>OxmlElement</code>)	Construit le fichier Word : titre, paragraphes, mise en gras. <code>OxmlElement</code> ajoute la balise XML <code>w:bidir</code> et <code>WD_ALIGN_PARAGRAPH.RIGHT</code> aligne à droite les paragraphes détectés comme arabes.
<code>fpdf2</code> (FPDF)	Construit le fichier PDF ligne par ligne (<code>multi_cell</code>) avec une police système (Tahoma) qui couvre à la fois le latin et l'arabe.
<code>arabic-reshaper</code>	Relie entre elles les lettres arabes (dans un PDF, sans ce traitement, chaque lettre s'affiche isolée au lieu d'être connectée à ses voisines comme à l'écrit).
<code>python-bidi</code> (<code>get_display</code>)	Réordonne le texte arabe de droite à gauche avant de l'écrire dans le PDF (le PDF ne gère pas nativement le sens d'écriture bidirectionnel).

Fonctions principales

- `contains_arabic(text)` — détecte si un message contient des caractères arabes, pour déclencher la mise en forme RTL adaptée.
- `build_docx(messages, sources)` — génère le fichier Word complet (titre, date, liste des articles, puis chaque échange question/réponse).
- `build_pdf(messages, sources)` — génère l'équivalent en PDF. La fonction interne `_write_line()` force le retour du curseur à la marge gauche après chaque ligne : par défaut, `fpdf2` le laisse collé à droite, ce qui provoquait une erreur dès la 2e ligne du document.